



DSA

Module 8

Searching

Author
Srinivas Dande



My **JAVA
LEARNING
CENTER**



8. Searching

- ◆ There are two Algorithms for searching an element
 - ✓ Linear Search
 - ✓ Binary Search

Linear Search

- ◆ Linear search is a sequential searching algorithm where we start from one end and check every element of the list until the required element is found.
- ◆ If the required element is found then we return the index of that element otherwise we return -1.
- ◆ It is the simplest searching algorithm.

Binary Search

- ◆ Binary Search can be implemented only on a Sorted list of items.
- ◆ If the elements are not sorted already, then we need to sort them first.
- ◆ In Binary Search, Element is always searched in the middle of a portion of an array and repeatedly dividing the search interval in half.
- ◆ If the required element is found then we return the index of that element otherwise we return -1.

8.1. Complexity

<u>Linear Search</u>	<u>Time Complexity</u>	<u>Aux Space Complexity</u>
With Iteration	$O(n)$	$O(1)$
With Recursion	$O(n)$	$O(n)$

<u>Binary Search</u>	<u>Time Complexity</u>	<u>Aux Space Complexity</u>
With Iteration	$O(\log n)$	$O(1)$
With Recursion	$O(\log n)$	$O(\log n)$



8.2.1. Linear Search using Iteration

Lab1.java

```
package com.jlcindia.searching;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {
    public static int linearSearch(int arr[],int element) {

        for(int i=0;i<arr.length;i++) {
            if(element == arr[i]){
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        int arr[] = {10,20,30,40,50,60,70};
        int element = 50;

        int result = linearSearch(arr,element);
        System.out.println(result);
    }
}
```



8.2.2. Linear Search using Recursion

Lab2.java

```
package com.jlcindia.searching;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {

    public static int linearSearch(int arr[],int element,int size) {

        if(size==0)
            return -1;

        if(arr[size-1]==element)
            return size-1;

        return linearSearch(arr, element, size-1);
    }

    public static void main(String[] args) {

        int arr[] = {10,20,30,40,50,60,70};
        int element = 50;
        int size=arr.length;

        int result = linearSearch(arr,element,size);
        System.out.println(result);
    }
}
```



8.3.1. Binary Search using Iteration

Lab3.java

```
package com.jlcindia.searching;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//LeetCode - 704
public class Lab3 {
    public static int binarySearch(int arr[],int element) {

        int low=0;
        int high=arr.length-1;

        while(low<=high) {
            int mid = (low+high)/2;

            if(element == arr[mid])
                return mid;
            else if(element > arr[mid])
                low = mid+1;
            else if(element < arr[mid])
                high = mid-1;
        }

        return -1;
    }
    public static void main(String[] args) {
        int arr[] = {10,20,30,40,50,60,70};
        int element = 25;

        int result = binarySearch(arr,element);
        System.out.println(result);
    }
}
```



8.3.2. Binary Search using Recursion

Lab4.java

```
package com.jlcindia.searching;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    public static int binarySearch(int arr[],int element,int low,int high) {
        if(low>high)
            return -1;

        int mid = (low+high)/2;
        if(element == arr[mid]) {
            return mid;
        }
        else if(element > arr[mid]) {
            low = mid+1;
            return binarySearch(arr,element,low,high);
        }
        else if(element < arr[mid]) {
            high = mid-1;
            return binarySearch(arr,element,low,high);
        }
        return -1;
    }
    public static void main(String[] args) {
        int arr[] = {10,20,30,40,50,60,70};
        int element = 10;
        int low = 0;          int high=arr.length-1;

        int result = binarySearch(arr,element,low,high);
        System.out.println(result);
    }
}
```



8.4. Problems on Binary Search

Problem1A.java

```
package com.jlcindia.searching;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// First Occurence of element in Sorted Array
public class Problem1A {

    public static int firstOccurence(int arr[],int element) {

        for(int i=0;i<arr.length;i++) {
            if(element == arr[i]){
                return i;
            }
        }

        return -1;
    }

    public static void main(String[] args) {

        int arr[] = {10,20,20,30,30,30,50};
        int element = 20;

        int result = firstOccurence(arr,element);
        System.out.println(result);

    }
}
```



Problem1B.java

```
package com.jlcindia.searching;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// First Occurence of element in Sorted Array

public class Problem1B {

    public static int firstOccurence(int arr[], int element) {

        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (element > arr[mid])
                low = mid + 1;
            else if (element < arr[mid])
                high = mid - 1;
            else if (element == arr[mid]) {
                if (mid == 0 || arr[mid - 1] != arr[mid])
                    return mid;
                else {
                    high = mid - 1;
                }
            }
        }

        return -1;
    }
}
```



```
public static void main(String[] args) {  
    int arr[] = { 10, 20, 20, 30, 30, 30, 50 };  
    int element = 20;  
    int result = firstOccurence(arr, element);  
    System.out.println(result);  
}  
}
```

Problem2A.java

```
package com.jlcindia.searching;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
//Last Occurence of element in Sorted Array  
  
public class Problem2A {  
    public static int lastOccurence(int arr[],int element) {  
        for(int i=arr.length-1;i>=0;i--) {  
            if(element == arr[i]){  
                return i;  
            }  
        }  
        return -1;  
    }  
    public static void main(String[] args) {  
        int arr[] = {10,20,20,30,30,30,50};  
        int element = 30;  
        int result = lastOccurence(arr,element);  
        System.out.println(result);  
    }  
}
```



Problem2B.java

```
package com.jlcindia.searching;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Last Occurence of element in Sorted Array
public class Problem2B {
    public static int lastOccurence(int arr[], int element) {
        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (element > arr[mid])
                low = mid + 1;
            else if (element < arr[mid])
                high = mid - 1;
            else if (element == arr[mid]) {
                if (mid == arr.length - 1 || arr[mid] != arr[mid + 1])
                    return mid;
                else {
                    low = mid + 1;
                }
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int arr[] = { 10, 20, 20, 30, 30, 30, 50 };
        int element = 30;
        int result = lastOccurence(arr, element);
        System.out.println(result);
    }
}
```



Problem3.java

```
package com.jlcindia.searching;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem-3: Count the Occurrences of Element in Sorted Array
//LeetCode -34: Find First and Last Position of Element in Sorted Array
public class Problem3 {
    public static int firstOccurrence(int arr[], int element) {
        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {

            int mid = (low + high) / 2;

            if (element < arr[mid]) {
                high = mid - 1;
            } else if (element > arr[mid]) {
                low = mid + 1;
            } else if (element == arr[mid]) {
                if (mid == 0 || arr[mid - 1] != arr[mid]) {
                    return mid;
                } else {
                    high = mid - 1;
                }
            }

        }

        return -1;
    }
}
```



```
public static int lastOccurrence(int arr[], int element) {

    int low = 0;
    int high = arr.length - 1;

    while (low <= high) {

        int mid = (low + high) / 2;

        if (element < arr[mid]) {
            high = mid - 1;
        } else if (element > arr[mid]) {
            low = mid + 1;
        } else if (element == arr[mid]) {
            if (mid == arr.length - 1 || arr[mid] != arr[mid + 1]) {
                return mid;
            } else {
                low = mid + 1;
            }
        }

    }

    return -1;
}
```

// Time Complexity - $O(\log n)$

```
public int[] searchRange(int arr[], int element) {
    int first = firstOccurrence(arr, element);
    if (first == -1)
        return new int[] {-1,-1};

    int last = lastOccurrence(arr, element);
    return new int[] {first,last};
}
```



// Time Complexity - $O(\log n)$

```
public static int countOccurrences(int arr[], int element) {
    int first = firstOccurrence(arr, element);
    if (first == -1)
        return 0;

    int last = lastOccurrence(arr, element);
    int count = last - first + 1;
    return count;
}

public static void main(String[] args) {
    int arr[] = { 20, 20, 20, 30, 30, 30, 30, 50, 50, 60 };
    // int arr[] = { 10, 10, 10, 10, 10, 10 };
    int element = 60;

    int resut = countOccurrences(arr, element);
    System.out.println(resut);
}
}
```



Problem4.java

```
package com.jlcindia.searching;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem-4: Sqrt(num)
//LetCode - 69
public class Problem4 {
    // Time Complexity - O(log n)
    public static long mySqrt(int num) {
        if (num < 2)
            return num;

        long low = 2;
        long high = num / 2;
        long result = 0;
        while (low <= high) {
            long mid = low + (high - low) / 2;
            result = mid * mid;
            if (result == num) {
                return mid;
            } else if (result < num) {
                low = mid + 1;
            } else if (result > num) {
                high = mid - 1;
            }
        }
        return high;
    }
    public static void main(String[] args) {
        int num = 36;
        long result = mySqrt(num);
        System.out.println(result);
    }
}
```



Problem5.java

```
package com.jlcindia.searching;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem-5: Find Peak Element
//LeetCode - 162
public class Problem5 {
    // Time Complexity - O(log n)
    public static int findPeakElement(int[] arr) {

        int low = 0;
        int high = arr.length - 1;

        while (low < high) {
            int mid = (low + high) / 2;

            if (arr[mid] > arr[mid + 1]) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }
        return low;
    }
    public static void main(String[] args) {
        // int arr[] = { 10, 11, 8, 2, 18, 20, 6 };
        // int arr[] = {1,2,3,4,5,6};
        // int arr[] = {6,5,4,3,2,1};
        int arr[] = {0,0,0,0,0};
        int result = findPeakElement(arr);
        System.out.println(result);
    }
}
```



Problem6.java

```
package com.jlcindia.searching;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem-6: . Search in Rotated Sorted Array
//LetCode - 33

public class Problem6 {

    // Time Complexity - O(log n)
    public static int search(int[] arr, int element) {

        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {

            int mid = (low + high) / 2;

            // 1. Mid is Matching
            if (element == arr[mid]) {
                return mid;
            }

            // 2. Mid's left Subarray Sorted
            if (arr[mid] >= arr[low]) {
                if (element >= arr[low] && element <= arr[mid]) {
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
        }
    }
}
```



```
        // 3. Mid's right Subarray is Sorted
        } else {
            if (element >= arr[mid] && element <= arr[high]) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
    }
    return -1;
}

public static void main(String[] args) {
    //    int arr[] = { 30, 40, 50, 60, 70, 10, 20 };
    int arr[] = { 60, 70, 10, 20, 30, 40, 50 };
    int element = 60;
    int result = search(arr, element);
    System.out.println(result);
}
}
```